

# Arrays and vectors with Numpy

## The power of packages

### Library (of packages)

- broadly refers to a reusable collection of code

### Matplotlib

- A library for creating static, animated, and interactive visualizations in Python

### Seaborn

- A visualization library based on matplotlib that provides a simpler interface for working with common plots and graphs

### Numpy (Numerical python)

- An essential library that contains multidimensional array and matrix data structures and functions to manipulate them
- Used for scientific computation

### Pandas (Python Data Analysis)

- A powerful library built on top of NumPy that's used to manipulate and analyze tabular data

### Module

- A simple Python file containing a collection of functions and global variables

## Introduction to NumPy

### Vectorization

- Enables operations to be performed on multiple components of a data object at the same time

## Import statement

- Uses the import keyword to load an external library, package, module, or function into your computing environment

## Aliasing

- Lets you assign an alternate name—or alias—by which you can refer to something

```
import numpy as np  
# in this case, np is an alias
```

Note that NP is the standard alias. If you use something else, other people might get confused when reading your code.

In NumPy, vectorization enables operations to be performed on multiple components of a data object at the same time. Data professionals often work with large datasets, and vectorized code helps them efficiently compute large quantities of data.

# Understand Python libraries, packages, and modules

Recently, you learned about Python libraries, packages, and modules. As you've discovered, importing these tools saves data professionals time and enhances their programming. Another benefit of commonly used libraries is that they are constantly scrutinized and updated by talented and knowledgeable programmers. Thus, you can be confident that the underlying code is high quality.

In this reading, you'll learn more about the basic features of libraries, packages, and modules; how they are related; and a selection of basic modules you might use as a data professional.

## Libraries, packages, and modules

A **library** is a corpus of reusable code modules and their accompanying documentation. Libraries are bundled into **packages** that you install, which can then be imported into your coding environment as needed. You'll typically encounter the terms "library" and "package" used interchangeably. Generally, this certificate program will refer to both as libraries, but it's important to be acquainted with both terms.

**Modules** are similar to libraries, in that they are groups of related classes and functions, but they are generally subcomponents of libraries. In other words, a library can have many different modules, and you can choose to import the entire library or just the module you need.

## Import statements

Libraries and modules beyond the Python standard library typically must be imported into your working environment on an as-needed basis. Additional libraries are installed first and then imported into your working environment as needed.

To import a library or module, use an import statement. Import statements require particular syntax using the **import** keyword. Here are some examples:

**Note:** The following code block is not interactive.

```
import numpy
```

This import statement imports the NumPy library into your working environment. After running this command, you'll have access to all NumPy classes and functions. For instance, to use the **array()** function on **[2, 4, 6]**, you'd write:

**Note:** The following code block is not interactive.

```
numpy.array([2, 4, 6])
```

In this case, the NumPy library is imported with the **np** alias. You can assign any abbreviation you like as an alias, but commonly used libraries have common aliases. Therefore, straying from those could cause confusion when sharing code with others. Here are some common libraries and their conventional aliases used by data professionals:

**Note:** The following code block is not interactive.

```
import matplotlib.pyplot as plt
```

You may have noticed that this syntax differs slightly from the other examples. In this case, `matplotlib` is the library and `pyplot` is a module inside. The `pyplot` module is aliased as **`plt`**, and it's accessed from the `matplotlib` library using the `dot`.

## Importing functions

Just as you can import libraries and modules, you can also import individual functions from libraries or from modules within libraries using a specific syntax. Here's an example depicting a common import when using the `scikit-learn` library to build machine learning models:

**Note:** The following code block is not interactive.

```
from sklearn.metrics import precision_score, recall_score
```

Again, notice the different syntax. The import statement begins with the **`from`** keyword, followed by **`sklearn.metrics`**—the `scikit-learn` library + the **`metrics`** module. Next is the **`import`** keyword followed by the desired functions. In this case, there are two: **`precision_score`** and **`recall_score`**.

The same syntax can be applied to the example using NumPy's **`array()`** function. However, note that you typically would not encounter individual functions being imported from NumPy. It's much easier and more common to just import the whole library.

**Note:** The following code block is not interactive.

```
from numpy import array
```

When a function is imported by name, like in this example, you can use it without any preceding syntax to indicate the library or module that it comes from:

**Note:** The following code block is not interactive.

```
array([2,4,6])
```

## Discouraged syntax

One last syntactical variation that you might encounter is:

**Note:** The following code block is not interactive.

```
from library.module import *
```

This imports everything from a particular library or module and allows you to use its functions without any preceding syntax. So, for instance, if you wrote **from numpy import \***, you'd be able to use all of NumPy's functions without preceding them with **numpy** or **np**. **This approach is not recommended** because it makes it difficult to track where functions come from. However, it's helpful to be aware of this because you will likely encounter it in your work as a data professional. And, in specific instances, it might be useful.

## Commonly used built-in modules

The Python standard library comes with a number of built-in modules relevant to data professional work such as **math**, **datetime**, and **random**. These can be imported without additional installation. In other words, you can import them directly, as long as you have Python installed. For example:

### datetime

- Provides many helpful date and time conversions and calculations

Example:

```
import datetime
date = datetime.date(1977, 5, 8)    # assign a date to a variable
print(date)                        # print date
print(date.year)                   # print the year that the date is in

delta = datetime.timedelta(days=30) # assign a timedelta of 30 days to a
                                     # variable
print(date - delta)                 # print date of 30 days prior

##
1977-05-08
1977
1977-04-08
```

## **math**

- Provides access to mathematical functions

Example:

```
import math
print(math.exp(0))      # e**0
print(math.log(1))      # ln(1)
print(math.factorial(4)) # 4!
print(math.sqrt(100))   # square root of 100

##
1.0
0.0
24
10.0
```

## **random**

- Useful for generating pseudo-random numbers (refer to the documentation for explanation of pseudo-random number generation)

Example:

```
import random
print(random.random())      # 0.0 <= X < 1.0
print(random.choice([1, 2, 3])) # choose a random element from a sequence
print(random.randint(1, 10))  # a <= X <= b

##
0.33337539511845016
1
1
```

## **Key takeaways**

Libraries, packages, and modules are gateways to Python's countless capabilities. Understanding how to leverage them for your own coding needs will unlock new tools and functions that make your work much more efficient. Check out the Python Package Index at the [PyPI](https://pypi.org/) repository to search for useful

libraries. There are packages designed for applications as diverse as chemistry, audio editing, natural language processing, and video games. Whatever it is you're trying to do, chances are someone has developed a suite of tools to help you!

## Python's new versions and features

You've learned that one of Python's great strengths is its dynamism: it is developed and supported by a large community of users. This makes it a language that changes and evolves. In other words, the language and its libraries are constantly growing, improving, and responding to user feedback.

Python's dynamism, while a strength, can introduce complexities. This can lead to challenges when code is intended to be used for long periods of time or when multiple people work on a project together. A script or program developed in one Python version is often tied to that particular environment, and running it in a different version can lead to unexpected behavior. This is because Python and its libraries are updated over time. So, if you developed a project that used, say, Python 2.7.4 in 2013, and you try to run it now in Python 3.12.4, there's a possibility that it won't execute the same way. Similarly, collaborative projects can be hindered by team members using different library versions, resulting in compatibility issues.

## Deprecation

Code inevitably evolves, gets phased out, and becomes obsolete. This process is known as deprecation. If you're working in a Jupyter notebook, you'll generally get a warning if something you use is scheduled to be deprecated in a future release. Your code will still run, but it might not continue to do so in future versions, or if it does, its behavior might change. It's best to pay attention to these warnings and change your code to conform with the coming change, because it's easier to do it when you're actively writing the code than to come back to it when it breaks and try to figure out what you were doing at that particular stage of the project. Also, small updates—for example, Python 3.11.8 to 3.11.9—won't be as disruptive as large updates—say, Python 2.X to 3.X, so you'll notice more deprecation when there are major releases.

It's important to note that you can still use older versions of Python and its libraries. You don't always have to update your code every time there is a new release. Indeed, older versions typically continue to receive support long after

they've been superseded by more recent releases. This is to allow for people and organizations to plan their code updates over time. While *you* might be able to simply update your project, it's a lot harder for a large company to update their code base while ensuring that nothing breaks.

## Environments

The particular set of conditions and configurations that you use to develop code is known as your environment. Your environment includes the tools, libraries, dependencies, settings, etc. that are used to write and run your code. So, when you work on a project with a group of people, it's important that you all share the same environment so the code runs the same way for everybody. If you're doing this course's labs on Coursera, you generally don't have to worry about the environment because it has been preconfigured for you. But if you're doing the labs off the Coursera platform, be aware that your code might execute differently than it would on Coursera, depending on your environment.

Environment management can be a complex and technical process, and we won't go into it much in this course because your environment has been preconfigured to allow you to focus on learning other skills. If you're interested in learning more about environment configuration or creating and working in your own development environment, [Anaconda Navigator](#) is a great place to get started. Anaconda Navigator is a free, open-source, downloadable graphical user interface that provides a user-friendly tool set to manage environments. It also has a suite of other built-in feature integrations like Jupyter notebooks, PyCharm, VSCode, and other popular tools used by data professionals.

## Check which versions you have

We try to keep the code in this course current by using recent stable releases of Python and relevant libraries, but updating everything in all the courses for every new release isn't feasible or necessary. If you're doing the Jupyter notebook activities on your own computer and not on Coursera, you may occasionally find differences in how your code runs compared to how it is demonstrated on Coursera. These differences are likely due to versioning.

To check which version of Python you're running in a Jupyter notebook, execute the following code in a notebook cell:

```
import sys  
print(sys.version)
```



To check which version of a library you're using, import the library and then use its `__version__` attribute to access its version information. For example:

```
import numpy as np
import pandas as pd
print(np.__version__)
print(pd.__version__)
```

## Key takeaways

Python evolves and grows in response to its users and use cases. This means that code updates can change the behavior of code that was written in older versions. Understanding how to assess, manage, and troubleshoot your coding environment is an important part of coding.

## Basic array operations

### N-dimensional array (ndarray)

- The core data object of Numpy
- The array is the core data structure of NumPy. The data object itself is known as an "n-dimensional array," or ndarray for short.
- The ndarray is vector
  - Recall that vectors enable many operations to be performed together when the code is executed, resulting in faster run-times that require less computer memory.
- If you want to change the size of the array, you have to reassign it

If I create an array with the integers one, two, and then a string of "coconut," NumPy will create an array that forces everything to the same data type, if possible.

```
arr = np.array([1, 2, 'coconut'])
arr
```

```
##  
array(['1', '2', 'coconut'], dtype='<U21')
```

In this case, everything becomes a string, represented here by "U21," meaning unicode 21.

## dtype

- We can use the dtype attribute if we want to check the data type of the contents of an array.

```
arr = np.array([1,2,4])  
arr.dtype  
  
##  
dtype('int64')
```

## shape

- We can use the shape attribute to confirm the shape of an array.

## ndim

- A NumPy attribute used to check the number of dimensions of an array

```
arr.shape  
  
##  
(3,)
```

```
arr.ndim  
##  
1
```

Data professionals will often need to confirm the shape and number of dimensions of their array.

```
array_2d = np.array([[1,2], [3,4], [5,6], [6,7]])
print(array_2d.shape)
print(array_2d.ndim)
print(array_2d)
```

```
##
(4, 2)
2
[[1 2]
 [3 4]
 [5 6]
 [6 7]]
```

```
arr_3d = np.array(
    [[[1,2],
      [3,4]],
     [[5,6],
      [7,8]],
     [[5,6],
      [7,8]]]
)
```

```
print(arr_3d.shape)
print(arr_3d.ndim)
arr_3d
##
(3, 2, 2)
3
array([[[1, 2],
        [3, 4]],
       [[5, 6],
        [7, 8]],
       [[5, 6],
        [7, 8]]])
```

## reshape()

- Numpy method used to change the shape of an array

```
print(array_2d)
array_2d.reshape(2,4) # → not an inplace function
print(array_2d)
reshaped_arr_2d = array_2d.reshape(2,4)
reshaped_arr_2d

##
[[1 2]
 [3 4]
 [5 6]
 [6 7]]
[[1 2]
 [3 4]
 [5 6]
 [6 7]]
array([[1, 2, 3, 4],
       [5, 6, 6, 7]])
```

## Math function

```
x = np.array([1, 2, 3, 4])
print(np.mean(x))
print(np.log(x)) #natural logarithm (ln or log_e())
print(np.floor(5.7))
print(np.ceil(5.3))
##
2.5
[0. 0.69314718 1.09861229 1.38629436]
5.0
6.0
```

# Reference guide: Arrays

As you've learned, NumPy is a powerful library capable of performing advanced numerical computing. One of its main benefits is the ability to work with arrays, as an operation applied to a vector executes much faster than the same operation applied to a list. Performance increases become further apparent when working with large volumes of data. This reading is a reference guide for working with NumPy arrays.

## Save this course item

You may want to save a copy of this guide for future reference. Use it as a resource for additional practice or in your future professional projects. To access a downloadable version of this course item, click the link below and select "Use Template."

[Reference guide: Arrays](#)

OR

If you don't have a Google account, download the item directly from the attachment below.

[Reference guide\\_ ArraysDOCX File](#)

## Create an array

As you've discovered, to use NumPy, you must first import it. Standard practice is to alias it as **np**.

### np.array()

This creates an **ndarray** (n-dimensional array). There is no limit to how many dimensions a NumPy array can have, but arrays with many dimensions can be more difficult to work with.

### 1-D array:

```
import numpy as np
array_1d = np.array([1, 2, 3])
array_1d

##
[1 2 3]
```

Notice that a one-dimensional array is similar to a list.

## 2-D array:

```
array_2d = np.array([(1, 2, 3), (4, 5, 6)])
array_2d

##
[[1 2 3]
 [4 5 6]]
```

Notice that a two-dimensional array is similar to a table.

## 3-D array:

```
array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
array_3d

##
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Notice that a three-dimensional array is similar to two tables.

## np.zeros()

- This creates an array of a designated shape that is pre-filled with zeros:

```
np.zeros((3, 2)) #create a 0's matrix with 3 rows and 2 columns

[[ 0.  0.]
 [ 0.  0.]
 [ 0.  0.]]
```

## np.ones()

- This creates an array of a designated shape that is pre-filled with one

```
np.ones((2, 2))
```

```
##  
[[ 1.  1.]  
 [ 1.  1.]]
```

## **np.full()**

- And this creates an array of a designated shape that is pre-filled with a specified value:

```
np.full((5, 3), 8)
```

```
[[ 8.  8.  8.]  
 [ 8.  8.  8.]  
 [ 8.  8.  8.]  
 [ 8.  8.  8.]  
 [ 8.  8.  8.]]
```

These functions are useful for various situations:

- To initialize an array of a specific size and shape, then fill it with values derived from a calculation
- To allocate memory for later use
- To perform matrix operations

## **Array methods**

NumPy arrays have many methods that allow you to manipulate and operate on them. For a full list, refer to the [NumPy array documentation](#). Some of the most commonly used methods follow:

### **ndarray.flatten()**

- This returns a copy of the array collapsed into one dimension.

```
array_2d = np.array([(1, 2, 3), (4, 5, 6)])
print(array_2d)
print()
array_2d.flatten()
```

```
[[1 2 3]
 [4 5 6]]

[1 2 3 4 5 6]
```

## **ndarray.reshape()**

- This gives a new shape to an array without changing its data.

```
array_2d = np.array([(1, 2, 3), (4, 5, 6)])
print(array_2d)
print()
array_2d.reshape(3, 2)
```

```
[[1 2 3]
 [4 5 6]]
```

```
[[1 2]
 [3 4]
 [5 6]]
```

Adding a value of -1 in the designated new shape makes the process more efficient, as it indicates for NumPy to automatically infer the value based on other given values.

```
array_2d = np.array([(1, 2, 3), (4, 5, 6)])
print(array_2d)
print()
array_2d.reshape(3, -1)
```

```
[[1 2 3]
```



```
[4 5 6]]
```

```
[[1 2]
```

```
[3 4]
```

```
[5 6]]
```

## **ndarray.tolist()**

- This converts an array to a list object. Multidimensional arrays are converted to nested lists.

```
array_2d = np.array([(1, 2, 3), (4, 5, 6)])
```

```
print(array_2d)
```

```
print()
```

```
array_2d.tolist()
```

```
[[1 2 3]
```

```
[4 5 6]]
```

```
[[1, 2, 3], [4, 5, 6]]
```

## **Mathematical functions**

NumPy arrays also have many methods that are mathematical functions:

- ndarray.max(): returns the maximum value in the array or along a specified axis.
- ndarray.mean(): returns the mean of all the values in the array or along a specified axis.
- ndarray.min(): returns the minimum value in the array or along a specified axis.
- ndarray.std(): returns the standard deviation of all the values in the array or along a specified axis.

```
a = np.array([(1, 2, 3), (4, 5, 6)])
```

```
print(a)
```

```
print()
```

```
print(a.max())
print(a.mean())
print(a.min())
print(a.std())
```

```
##
[[1 2 3]
 [4 5 6]]
```

```
6
3.5
1
1.70782512766
```

## Array attributes

NumPy arrays have several attributes that enable you to access information about the array. Some of the most commonly used attributes include the following:

- `ndarray.shape`: returns a tuple of the array's dimensions.
- `ndarray.dtype`: returns the data type of the array's contents.
- `ndarray.size`: returns the total number of elements in the array.
- `ndarray.T`: returns the array transposed (rows become columns, columns become rows).

```
array_2d = np.array([(1, 2, 3), (4, 5, 6)])
print(array_2d)
print()
```

```
print(array_2d.shape)
print(array_2d.dtype)
print(array_2d.size)
print(array_2d.T)
```

```
##
[[1 2 3]
```

```
[4 5 6]]
```

```
(2, 3)
```

```
int64
```

```
6
```

```
[[1 4]
```

```
[2 5]
```

```
[3 6]]
```

## Indexing and slicing

Access individual elements of a NumPy array using indexing and slicing.

Indexing in NumPy is similar to indexing in Python lists, except multiple indices can be used to access elements in multidimensional arrays.

```
a = np.array([(1, 2, 3), (4, 5, 6)])
```

```
print(a)
```

```
print()
```

```
print(a[1])
```

```
print(a[0, 1])
```

```
print(a[1, 2])
```

```
##
```

```
[[1 2 3]
```

```
[4 5 6]]
```

```
[4 5 6]
```

```
2
```

```
6
```

Slicing may also be used to access subarrays of a NumPy array:

```
a = np.array([(1, 2, 3), (4, 5, 6)])
```

```
print(a)
```

```
print()
```

```
a[:, 1:]
```

```
##  
[[1 2 3]  
 [4 5 6]]  
  
[[2 3]  
 [5 6]]
```

## Array operations

NumPy arrays support many operations, including mathematical functions and arithmetic. These include array addition and multiplication, which performs element-wise arithmetic on arrays:

```
a = np.array([(1, 2, 3), (4, 5, 6)])  
b = np.array([[1, 2, 3], [1, 2, 3]])  
print('a:')  
print(a)  
print()  
print('b:')  
print(b)  
print()  
print('a + b:')  
print(a + b)  
print()  
print('a * b:')  
print(a * b)
```

```
##  
a:  
[[1 2 3]  
 [4 5 6]]  
  
b:  
[[1 2 3]  
 [1 2 3]]  
  
a + b:
```

```
[[2 4 6]
 [5 7 9]]

a * b:
[[ 1  4  9]
 [ 4 10 18]]
```

In addition, there are nearly 100 other useful [mathematical functions](#) that can be applied to individual or multiple arrays.

## Mutability

NumPy arrays are mutable, but with certain limitations. For instance, an existing element of an array can be changed:

```
a = np.array([(1, 2), (3, 4)])
print(a)
print()

a[1][1] = 100
a

##
[[1 2]
 [3 4]]

[[ 1  2]
 [ 3 100]]
```

However, the array cannot be lengthened or shortened:

```
a = np.array([1, 2, 3])
print(a)
print()

a[3] = 100
a

##
```

Error on line 5:

```
a[3] = 100
```

IndexError: index 3 is out of bounds for axis 0 with size 3

## How NumPy arrays store data in memory

NumPy arrays work by **allocating a contiguous block of memory at the time of instantiation**. Most other structures in Python don't do this; their data is scattered across the system's memory. This is what makes NumPy arrays so fast; all the data is stored together at a particular address in the system's memory.

Interestingly, this is also what prevents an array from being lengthened or shortened: The abutting memory is occupied by other information. There's no room for more data at that memory address. However, existing elements of the array can be replaced with new elements.



The only way to lengthen an array is to copy the existing array to a new memory address along with the new data.

## Task 3: Calculate percentage of readings with cleanest AQI

You are interested in how many air quality readings in the data represent the cleanest air, which we'll consider **readings of 5 or less**.

To perform this calculation, you'll make use of one of the properties of arrays that make them so powerful: their element-wise operability. For example, when you add an integer to an `ndarray` using the `+` operator, it performs an element-wise addition on the whole array.

```
[IN] my_array = np.array([1, 2, 3])
      my_array = my_array + 10
      print(my_array)
```

```
[OUT] [11, 12, 13]
```

**The same concept applies to comparison operators used on an `ndarray`.** With this in mind:

- Calculate the percentage of AQI readings that are considered cleanest:

1. Use a comparison statement to get an array of Boolean values that is the same length as `aqi_array`. Assign the result to variable called `boolean_aqi`.
2. Calculate the number of `True` values in the `boolean_aqi` and divide this number by the total number of values in the array. Assign the result to a variable named `percent_under_6` and print it.

*Expected result:*

```
[OUT] 0.3194202898550725
```

Solution

```
# 1. ### YOUR CODE HERE
aqi_array = np.array(aqi_list)

# 1. ### YOUR CODE HERE ###
boolean_aqi = (aqi_array <= 5)
print(boolean_aqi)
# 2. ### YOUR CODE HERE ###
percent_under_6 = boolean_aqi.sum() / len(boolean_aqi)
percent_under_6
```